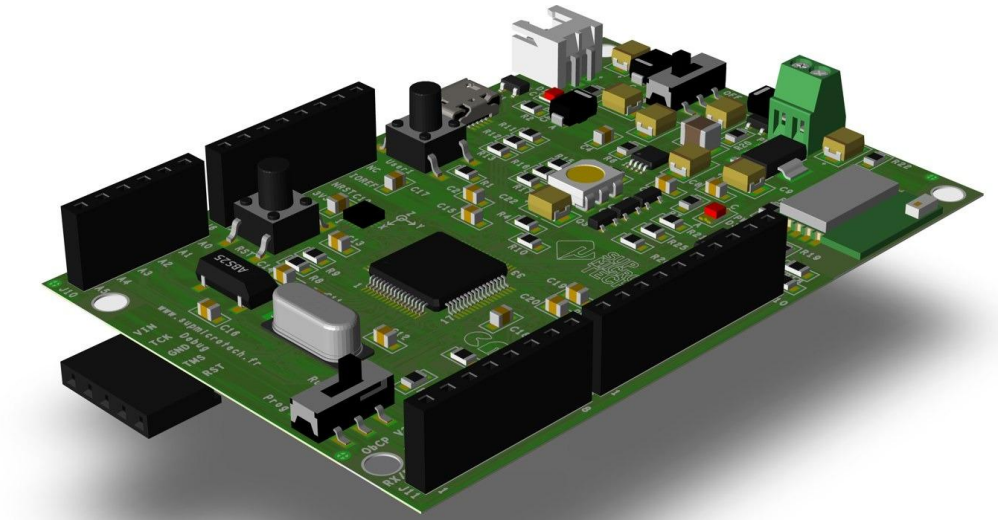
ObCP OTA BLE

Charger un programme dans l'ObCP par Bluetooth



Guide d'utilisation et description du programme

Chargeur BLE 1.1 · application PC obcp_ota 1.2 · protocole OTA version 1

SUPMICROTECH-ENSMM — option CROC

Novembre 2026

Sommaire

1. Présentation 4
1.1 Pourquoi charger un programme par Bluetooth ? 4
1.2 L'essentiel en quatre points 4
1.3 Vue d'ensemble 4

2. Contenu du dossier 6

3. Installation (une seule fois) 7
3.1 Sur le PC : Python et Bleak 7
3.2 Programmer le chargeur dans l'ObCP 7
3.3 Importer les projets dans STM32CubeIDE 8
3.4 Choisir sa carte en salle de TP 8

4. Utilisation au quotidien 10
4.1 Mettre la carte en mode chargement 10
4.2 Compiler et transférer depuis STM32CubeIDE 10
4.3 Transférer en ligne de commande 12
4.4 Revenir à l'application sans transférer 12
4.5 Prise en main avec la démonstration 12
4.6 Si la liaison Bluetooth est coupée 13

5. Rendre une application compatible 14
5.1 La modification indispensable : le script d'édition de liens 14
5.2 Recommandé : indiquer la table des interruptions 14
5.3 Facultatif : revenir au chargeur par logiciel 15
5.4 Vérifier le projet 16
5.5 Règles à respecter 16

6. Compatibilité avec STM32CubeMX 17
6.1 Principe 17
6.2 Régénérer le code 17
6.3 Réglages CubeMX du chargeur à conserver 18

7. Fonctionnement détaillé 19
7.1 Carte mémoire 19
7.2 Le démarrage : application ou chargeur ? 19
7.3 Le service Bluetooth « ObCP OTA » 21
7.4 Le protocole de transfert 21
7.5 Écriture en flash et sûreté de fonctionnement 23
7.6 Contrôle de flux et débit 24
7.7 Nom et adresse Bluetooth de la carte 24
7.8 Fin du transfert et coupures de liaison 25

8. Description du programme 26
8.1 Le chargeur 26
8.2 L'application de démonstration 27
8.3 L'application PC 27
8.4 Lire le code : quelques notions de C 28

9. Tests et état de validation 29
9.1 Ce qui a été vérifié 29
9.2 Ce qui reste à essayer sur une vraie ObCP 29
9.3 Procédure de premiers essais 30
9.4 Relancer les tests 30
10. Dépannage 31
11. Pour aller plus loin 32
Annexe A – Glossaire 33
Annexe B – Référence des fonctions 34
B.1 Chargeur et démonstration (C) 34
B.2 Application PC (Python) 37

1. Présentation

1.1 Pourquoi charger un programme par Bluetooth ?

L'ObCP (Objet Connecté Personnel) n'a pas de sonde de programmation ST-LINK. Jusqu'ici, un nouveau programme se chargeait par l'USB, grâce au chargeur DFU gravé par ST dans le STM32 : il fallait placer l'interrupteur SW4 sur « Prog », appuyer sur RESET, programmer avec STM32CubeProgrammer, puis remettre SW4 sur « Run » et appuyer de nouveau sur RESET.

Le chargement par Bluetooth, appelé **OTA** (*over the air*, « par les ondes »), supprime ces manipulations : depuis STM32CubeIDE, une seule commande compile le programme et l'envoie à la carte, qui redémarre aussitôt sur la nouvelle version. La carte peut rester sur sa batterie, dans son boîtier, sans câble de données.

C'est aussi le procédé qu'utilisent les objets connectés du commerce (montres, capteurs, ampoules) pour recevoir leurs mises à jour. Le projet permet donc de découvrir concrètement la mémoire flash, le démarrage d'un microcontrôleur, le Bluetooth Low Energy et la conception d'un protocole de transfert fiable.

1.2 L'essentiel en quatre points

1. La flash du STM32L476RG (1 Mo) est partagée : le chargeur occupe la **banque 1** (à partir de `0x08000000`), l'application la **banque 2** (à partir de `0x08080000`).
2. Pour passer en **mode chargement** : maintenir le bouton **USER** et appuyer brièvement sur **RESET**. La LED bleue clignote alors une fois par seconde. Si le programme de la carte connaît la commande `#ota` (programme modèle 2026, version 2.1), l'application PC s'en charge elle-même.
3. Pour rendre une application compatible, **une seule ligne** change dans son script d'édition de liens (chapitre 5).
4. Le transfert est vérifié de bout en bout (numéros de paquets, CRC-32, table des vecteurs) : un transfert interrompu ne lance jamais un programme incomplet, et le chargeur lui-même n'est jamais effacé.

1.3 Vue d'ensemble

Le système comprend trois programmes (figure 1) :

- **le chargeur BLE** (projet `ObCP_BLE_Loader`), installé une fois pour toutes au début de la mémoire flash du STM32. À chaque démarrage, il décide de lancer l'application ou d'attendre un nouveau programme par Bluetooth ;
- **l'application** : n'importe quel programme STM32CubeIDE, par exemple la démonstration `ObCP_OTA_Demo` ou le programme modèle de l'ObCP. Elle est rangée dans la seconde moitié de la flash ;
- **l'application PC** `obcp_ota.py`, écrite en Python, qui lit le fichier compilé et l'envoie à la carte par le Bluetooth du PC. STM32CubeIDE la lance par un simple clic.

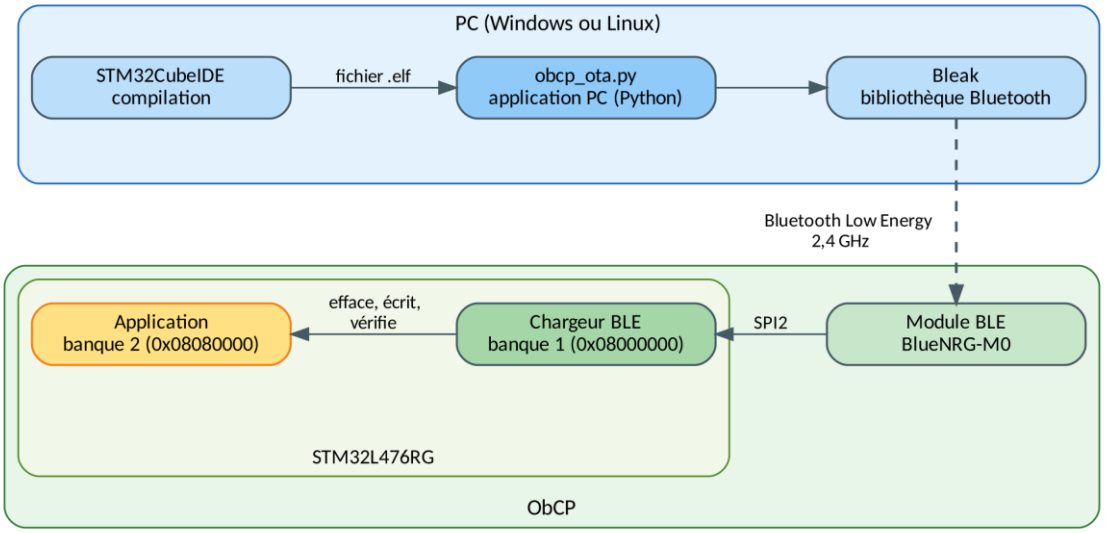


Figure 1 – Les trois programmes et leurs liaisons

2. Contenu du dossier

Le dossier `ObCP_OTA_BLE` réunit trois projets et les outils associés :

```
ObCP_OTA_BLE/
├── LISEZMOI.md                    présentation rapide
├── Documentation/                 ce guide et les sources de ses figures
├── Binaires/                      programmes déjà compilés (configuration Release)
├── ObCP_BLE_Loader/               projet du CHARGEUR (STM32CubeIDE + CubeMX)
│   ├── ObCP_BLE_Loader.ioc        configuration CubeMX
│   ├── Core/                      fichiers générés par CubeMX (main.c, gpio.c...)
│   ├── BlueNRG-MS/, Middlewares/  bibliothèque Bluetooth de ST (X-CUBE-BLE1)
│   ├── Drivers/                   bibliothèque HAL du STM32L4
│   └── Loader/                    code du chargeur : Inc/ (.h) et Src/ (.c)
├── ObCP_OTA_Demo/                 projet de l'APPLICATION de démonstration
│   ├── ObCP_OTA_Demo.ioc
│   ├── Core/Src/demo.c            la démonstration
│   └── Core/Inc/obcp_ota_request.h   fonctions OTA à copier dans vos projets
├── ObCP_OTA_Tools/                APPLICATION PC et outils pour STM32CubeIDE
│   ├── obcp_ota.py                programme principal
│   ├── ota_image.py               lecture des fichiers .elf, .hex, .bin
│   ├── ota_transfert.py           algorithme du transfert
│   ├── ota_protocole.py           constantes du protocole
│   ├── verif_cubemx.py            vérificateur de projet
│   └── ObCP OTA - *.launch        commandes du menu External Tools
└── Tests/                         tests sur PC, sans carte
```

Le dossier `Binaires` contient des programmes prêts à l'emploi, compilés en configuration Release :

Fichier	Contenu	Usage
`ObCP_Chargeur_et_Demo.hex`	chargeur + démonstration (LED verte)	première programmation, en DFU (§ 3.2)
`ObCP_BLE_Loader.hex` / `.elf`	chargeur seul	programmation en DFU, sans application
`ObCP_OTA_Demo_v1.elf` / `.hex`	démonstration, LED verte	transfert par Bluetooth
`ObCP_OTA_Demo_v2.elf` / `.hex`	démonstration, LED bleue	second transfert, pour voir le changement

3. Installation (une seule fois)

3.1 Sur le PC : Python et Bleak

L'application PC fonctionne sous Windows 10 et 11 et sous Linux, avec le Bluetooth intégré du PC ou une clé Bluetooth 4.0 (ou plus récente).

1. Installer **Python 3.10** ou plus récent depuis python.org. Sous Windows, cocher **Add python.exe to PATH** dans la première fenêtre de l'installateur.
2. Dans un terminal (invite de commandes sous Windows), installer la bibliothèque Bluetooth **Bleak** : `python -m pip install bleak`
3. Vérifier l'installation : `python ObCP_OTA_Tools/obcp_ota.py --version` doit afficher `obcp_ota 1.2`.

> **À savoir :** sous Linux, la commande s'appelle souvent `python3`. Installez le paquet python-is-python3 (`sudo apt install python-is-python3`) : il crée la commande `python`, utilisée par les commandes de STM32CubeIDE (§ 3.3).

3.2 Programmer le chargeur dans l'ObCP

Le chargeur s'installe par l'USB, en mode DFU, comme n'importe quel programme de l'ObCP. Le plus simple est de programmer directement `Binaires/ObCP_Chargeur_et_Demo.hex`, qui contient le chargeur et la démonstration.

1. Relier l'ObCP au PC par un câble USB **de données**, placer **SW4 sur Prog** et appuyer sur **RESET** (SW3).
2. Dans STM32CubeProgrammer : interface **USB**, port **USB1**, **Connect**. Onglet *Erasing & programming* : **Full chip erase**, puis choisir le fichier `ObCP_Chargeur_et_Demo.hex`, cocher *Verify programming* et *Run after programming*, puis **Start Programming**.
3. Remettre **SW4 sur Run** et appuyer sur **RESET** : la LED clignote en vert, la démonstration fonctionne.

En ligne de commande, l'étape 2 s'écrit :

```
STM32_Programmer_CLI -c port=usb1 -e all -w Binaires/ObCP_Chargeur_et_Demo.hex -v -s
```

> **Attention :** l'effacement complet (*Full chip erase*, option `-e all`) efface aussi une éventuelle application : c'est voulu pour une première installation.
>
> Ensuite, ne programmez plus l'ObCP en DFU avec un programme ordinaire (lié à `0x08000000`) : il écraserait le chargeur. Pour revenir à l'ancienne méthode, il suffit de reprogrammer le chargeur.

Pour installer le chargeur seul, programmer `ObCP_BLE_Loader.hex` (ou compiler le projet `ObCP_BLE_Loader` et programmer son fichier .elf). Sans application, le chargeur reste actif : la LED clignote en violet.

Mettre à jour le chargeur (par exemple de la version 1.0 à la version 1.1) se fait exactement de la même façon, en DFU. L'effacement complet supprime l'application installée : avec `ObCP_Chargeur_et_Demo.hex`, la démonstration est réinstallée en même temps. L'application PC affiche la version du chargeur à chaque connexion.

3.3 Importer les projets dans STM32CubeIDE

1. *File › Import › General › Existing Projects into Workspace*, choisir le dossier `ObCP_OTA_BLE`, cocher **ObCP_OTA_Tools** et **ObCP_OTA_Demo** (et **ObCP_BLE_Loader** pour modifier le chargeur), puis **Finish**.
2. Le projet `ObCP_OTA_Tools` n'est pas un programme pour la carte : il contient l'application PC et trois commandes prêtes à l'emploi, qui apparaissent dans le menu **Run › External Tools** (bouton à droite du bouton Run, avec une valise rouge).

Commande	Rôle
ObCP OTA - compiler et transferer	compile le projet sélectionné, puis envoie son programme à la carte
ObCP OTA - choisir ma carte	liste les cartes en mode chargement et mémorise la vôtre (§ 3.4)
ObCP OTA - verifier le projet	vérifie qu'un projet est prêt pour le chargement par Bluetooth (§ 5.4)

> **À savoir :** si les commandes n'apparaissent pas directement dans le menu, ouvrir *Run › External Tools › External Tools Configurations…* : elles sont rangées sous *Program*, et le bouton **Run** les lance. On peut les ajouter au menu dans l'onglet *Common* (case *External Tools*).

> **Astuce :** ces commandes lancent Python grâce à la variable `${system_path:python}`, qui cherche Python dans le PATH. Si Python n'y est pas, ouvrir la configuration et remplacer le champ *Location* par le chemin complet de `python.exe`, par exemple `C:\Users\nom\AppData\Local\Programs\Python\Python312\python.exe`.

Les trois commandes sont décrites dans des fichiers `.launch` du projet `ObCP_OTA_Tools`. Pour mémoire, leurs réglages sont les suivants (onglets *Main* et *Build* de la configuration) :

Champ	Valeur pour « compiler et transferer »
Location	`${system_path:python}`
Working Directory	`${workspace_loc:/ObCP_OTA_Tools}`
Arguments	`"${workspace_loc:/ObCP_OTA_Tools/obcp_ota.py}" "${project_loc}"`
Build before launch	*The project containing the selected resource*
Environnement	`PYTHONIOENCODING=utf-8`, `PYTHONUNBUFFERED=1` ; console en UTF-8

3.4 Choisir sa carte en salle de TP

Toutes les cartes en mode chargement s'annoncent sous un nom de la forme **ObCP-OTA-A1B2**, où A1B2 sont les quatre derniers chiffres de l'adresse Bluetooth propre à chaque carte. Pour être sûr de viser la sienne :

1. Mettre sa carte en mode chargement (§ 4.1).
2. Lancer **ObCP OTA - choisir ma carte** (ou `python obcp_ota.py --liste`) : les cartes trouvées s'affichent, la plus proche en premier.
3. Taper le numéro de sa carte, puis Entrée : la LED de cette carte clignote en blanc pendant 3 s. Si c'est bien la vôtre, c'est terminé : les transferts suivants la viseront automatiquement.

```
ObCP OTA 1.2 - chargement d'un programme par Bluetooth
Recherche des cartes en mode chargement (5 s)...
      1. ObCP-OTA-A1B2   signal -48 dBm
      2. ObCP-OTA-77C0   signal -71 dBm
      Numéro de VOTRE carte (Entrée = ne rien changer) : 1
      ObCP-OTA-A1B2 est mémorisée : les prochains transferts la viseront.
      Sa LED clignote en blanc pendant 3 s : vérifiez que c'est bien la vôtre !
```

Choix de la carte dans la console de STM32CubeIDE (cliquer dans la console pour taper la réponse)

Le choix est enregistré dans le fichier `.obcp_ota.json` du dossier personnel de l'utilisateur, avec l'adresse Bluetooth du chargeur de la carte (depuis la version 1.2) : elle permet de reconnaître la carte quand elle fonctionne normalement (§ 4.2). Seul avec sa carte, on peut se passer de cette étape : si une seule carte est en mode chargement, elle est choisie et mémorisée automatiquement. Si plus tard une autre carte se présente seule, l'application PC demande confirmation avant de l'utiliser.

4. Utilisation au quotidien

4.1 Mettre la carte en mode chargement

Le chargeur reste actif, au lieu de lancer l'application, dans trois situations :

Situation	Comment	LED
Bouton	maintenir **USER**, appuyer brièvement sur **RESET**, relâcher USER	bleue
Demande de l'application	par exemple un appui long (2 s) sur USER dans la démonstration, ou la commande `#ota` du programme modèle 2026 (§ 5.3), que l'application PC sait envoyer elle-même (§ 4.2)	bleue
Aucune application valide	carte neuve, transfert interrompu…	violette

La LED RVB indique ensuite l'état du chargeur :

LED	Signification
bleue, bref éclair chaque seconde	attente d'une connexion ; une application est installée
violette, bref éclair chaque seconde	attente d'une connexion ; aucune application valide
bleue fixe	PC connecté
verte, change à chaque kilo-octet	transfert en cours
verte fixe	programme reçu, vérifié et validé
rouge fixe	erreur de transfert (le message est affiché sur le PC)
blanche clignotante (3 s)	identification demandée par le PC (§ 3.4)
rouge, clignotement rapide	le module Bluetooth ne répond pas (chapitre 10)

4.2 Compiler et transférer depuis STM32CubeIDE

1. Mettre la carte en mode chargement (§ 4.1), sauf si elle a déjà reçu un programme depuis ce PC et que son programme connaît la commande `#ota` : l'application PC l'y met alors elle-même (voir plus bas). On peut aussi le faire juste après l'étape 3 : l'application PC attend la carte pendant 30 s.
2. Sélectionner le projet dans *Project Explorer*, ou ouvrir l'un de ses fichiers dans l'éditeur.
3. Lancer **Run › External Tools › ObCP OTA - compiler et transferer**.

STM32CubeIDE compile le projet (comme le marteau), puis l'application PC prend le fichier .elf le plus récent du projet et l'envoie. La console affiche les quatre étapes :

```
ObCP OTA 1.2 - chargement d'un programme par Bluetooth
[1/4] Programme : C:\STM32\ObCP_OTA_Demo\Debug\ObCP_OTA_Demo.elf
      format ELF, créé le 02/11/2026 10:42:05
      7 032 octets à partir de 0x08080000, CRC-32 0x25D6BA90
[2/4] Recherche de la carte ObCP-OTA-A1B2...
      ObCP-OTA-A1B2 trouvée (signal -52 dBm)
[3/4] Connexion...
      chargeur version 1.1, application déjà installée : oui
      mode chargement activé par : bouton USER maintenu pendant le démarrage
[4/4] Transfert
      effacement de 4 pages de flash...
      [------------------------------]   0 %  0,0/6,9 Ko  0,0 Ko/s
      [########----------------------]  29 %  2,0/6,9 Ko  6,7 Ko/s
      [##################------------]  58 %  4,0/6,9 Ko  6,8 Ko/s
      [##############################] 100 %  6,9/6,9 Ko  6,8 Ko/s
      vérification par l'ObCP : CRC-32 0x25D6BA90 correct
Terminé : 7 032 octets en 1,2 s (5,7 Ko/s).
L'ObCP redémarre sur le nouveau programme.
```

Exemple de sortie dans la console (débit de l'ordre de celui mesuré lors d'un premier essai sur une ObCP : 6,8 Ko/s)

Avant d'effacer quoi que ce soit sur la carte, l'application PC vérifie le programme : s'il est lié à la mauvaise adresse (chapitre 5), elle refuse de l'envoyer et explique la modification à faire. Si un fichier source est plus récent que le .elf, c'est que la compilation a échoué : elle s'arrête aussi, pour ne pas envoyer l'ancien programme par erreur.

Carte en fonctionnement : la commande « #ota » envoyée par l'application PC

Depuis la version 1.2, l'application PC reconnaît la carte mémorisée même quand elle fonctionne normalement (son programme annonce alors sous son propre nom, OBCP-CROC pour le programme modèle). Elle se connecte à ce programme, lui envoie la commande `#ota` comme on la taperait dans un terminal, puis attend que la carte se présente en mode chargement, 15 s au plus. La suite est identique :

```
[2/4] Recherche de la carte ObCP-OTA-A1B2...
      ObCP-OTA-A1B2 fonctionne normalement : envoi de la commande #ota...
      la carte redémarre dans son chargeur...
      ObCP-OTA-A1B2 trouvée (signal -55 dBm)
[3/4] Connexion...
      chargeur version 1.1, application déjà installée : oui
      mode chargement activé par : demande de l'application (commande #ota...)
```

La carte fonctionnait : l'application PC l'a fait passer elle-même en mode chargement

Il faut pour cela que :

- la carte ait déjà été jointe dans son chargeur depuis ce PC avec la version 1.2 : l'adresse Bluetooth de son chargeur est alors mémorisée (§ 3.4), et celle de son programme s'en déduit (§ 7.7) ;
- son programme connaisse la commande `#ota` : c'est le cas du programme modèle OBCP_template_2026 à partir de sa version 2.1.0 (§ 5.3), pas de la démonstration ni du programme modèle de 2024 ;
- aucun téléphone ne soit connecté à la carte : une carte connectée n'annonce plus ;
- le système fournisse les adresses Bluetooth : c'est le cas sous Windows et sous Linux, pas sous macOS.

Sinon, rien ne change : l'application PC attend la carte en mode chargement, comme avant. Elle n'envoie jamais `#ota` à une autre carte que la sienne : en salle de TP, les programmes de toutes les cartes peuvent porter le même nom, mais pas la même adresse.

4.3 Transférer en ligne de commande

Depuis le dossier `ObCP_OTA_Tools` :

```
python obcp_ota.py ..\ObCP_OTA_Demo                  projet : son .elf le plus récent
python obcp_ota.py ..\Binaires\ObCP_OTA_Demo_v2.elf  fichier .elf, .hex ou .bin
python obcp_ota.py programme.elf --carte A1B2        vise une carte précise
python obcp_ota.py --liste                           choisit sa carte (§ 3.4)
```

Option	Effet
source	fichier .elf, .hex ou .bin, ou dossier d'un projet STM32CubeIDE
`--carte NOM`	carte à utiliser (A1B2 ou ObCP-OTA-A1B2). Sans cette option : la carte mémorisée, ou la seule présente
`--liste`	cherche les cartes en mode chargement et mémorise la vôtre
`--oublier`	oublie la carte mémorisée (et l'adresse de son chargeur)
`--sans-lancer`	ne redémarre pas la carte après le transfert (appui court sur USER pour lancer)
`--attente S`	temps laissé pour trouver la carte, en secondes (30 par défaut)
`--forcer`	envoie le .elf même s'il est plus ancien que les sources
`--version`, `-h`	version, aide

Le programme renvoie le code 0 si le transfert a réussi et 1 sinon : il peut donc servir dans un script. Un fichier .bin ne contient pas d'adresse : il est supposé commencer à `0x08080000`. Préférez le .elf, qui permet toutes les vérifications.

4.4 Revenir à l'application sans transférer

- un **appui court sur USER** dans le chargeur relance l'application (si elle est valide) ;
- au bout de **2 minutes sans connexion**, le chargeur relance l'application de lui-même ;
- un appui sur **RESET seul** (sans USER) lance toujours l'application.

4.5 Prise en main avec la démonstration

1. Transférer le projet `ObCP_OTA_Demo` (§ 4.2) : la LED clignote en vert.
2. Appui court sur USER : la vitesse du clignotement change.
3. Dans `Core/Src/demo.c`, remplacer `#define DEMO_VERSION 1` par `#define DEMO_VERSION 2` et enregistrer.
4. Appui long (2 s) sur USER : la LED devient blanche, puis la carte passe en mode chargement (LED bleue).
5. Relancer **ObCP OTA - compiler et transferer** : la LED clignote maintenant en bleu. Le nouveau programme a remplacé l'ancien, sans câble ni interrupteur.

4.6 Si la liaison Bluetooth est coupée

Une coupure de la liaison (carte trop loin, obstacle, PC qui se met en veille…) n'abîme jamais la carte : un programme incomplet n'est jamais lancé (§ 7.5). Depuis la version 1.1, l'application PC détecte la coupure aussitôt, se reconnecte d'elle-même (deux fois au plus) et relit l'état de la carte :

- si la coupure a eu lieu juste après la commande END et que la carte indique un programme installé, c'est qu'elle l'avait **vérifié (CRC) et validé** : le transfert est terminé, l'application PC envoie simplement RUN ;
- sinon, le transfert recommence au début, de façon automatique.

Avec un chargeur 1.1, la carte indique aussi la cause de la coupure, telle qu'elle l'a constatée :

```
      [##############################] 100 %  6,9/6,9 Ko  6,8 Ko/s
      connexion Bluetooth perdue ([WinError -2147023673] …) ; nouvelle connexion (2/3)...
      la carte indique : plus aucun échange radio : carte trop loin, obstacle,
      perturbation, ou carte qui a redémarré (code 0x08)
      la carte avait vérifié (CRC) et validé le programme avant la coupure
Terminé : 7 032 octets en 2,9 s (2,4 Ko/s).
L'ObCP redémarre sur le nouveau programme.
```

Coupure juste après END : le programme était déjà validé, aucun second envoi n'est nécessaire

Si la carte n'est plus visible après la coupure, l'application PC s'arrête avec un message : la carte a peut-être déjà redémarré sur le nouveau programme (s'il fonctionne, c'est qu'il a été validé) ; sinon, il suffit de la remettre en mode chargement et de relancer le transfert.

5. Rendre une application compatible

N'importe quel projet STM32CubeIDE pour l'ObCP peut être chargé par Bluetooth. L'application n'a besoin d'aucun code Bluetooth particulier : c'est le chargeur qui reçoit le programme et l'écrit en flash. Une seule modification est indispensable.

5.1 La modification indispensable : le script d'édition de liens

L'éditeur de liens (*linker*) place le programme en mémoire d'après le fichier `STM32L476RGTX_FLASH.ld`, à la racine du projet. Par défaut, il range le programme au début de la flash, à `0x08000000`… c'est-à-dire à la place du chargeur. Il faut donc le déplacer en banque 2 :

1. Ouvrir `STM32L476RGTX_FLASH.ld` (double-clic dans *Project Explorer* ; s'il s'ouvre dans un éditeur graphique, faire clic droit › *Open With* › *Text Editor*).
2. Dans le bloc `MEMORY`, remplacer la ligne `FLASH` :

```
MEMORY
{
  RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH = 96K
  RAM2    (xrw)    : ORIGIN = 0x10000000,   LENGTH = 32K
  FLASH    (rx)    : ORIGIN = 0x8000000,   LENGTH = 1024K    <- avant
  FLASH    (rx)    : ORIGIN = 0x8080000,   LENGTH = 512K     <- après
}
```

3. Enregistrer, puis recompiler.

Toutes les sections du programme (table des vecteurs, code, constantes, valeurs initiales des variables) se retrouvent alors à partir de `0x08080000`, l'adresse où le chargeur va les chercher. Le fichier `STM32L476RGTX_RAM.ld` n'est pas utilisé : rien à y changer.

> **À savoir :** si vous oubliez cette étape, rien n'est cassé : `obcp_ota.py` refuse d'envoyer un programme lié à `0x08000000` et rappelle la modification à faire. En principe, STM32CubeMX ne réécrit pas un script d'édition de liens existant ; l'outil de vérification (§ 5.4) contrôle de toute façon cette ligne après chaque génération du code.

5.2 Recommandé : indiquer la table des interruptions

Chaque programme commence par sa **table des vecteurs** : la liste des adresses des fonctions à exécuter à chaque interruption (SysTick, EXTI…). Le registre **VTOR** du processeur indique où se trouve cette table. Après un reset, il vaut 0, c'est-à-dire la table… du chargeur.

Le chargeur règle VTOR sur `0x08080000` avant de lancer l'application : une application fonctionne donc même sans rien ajouter. Il est cependant recommandé de le faire aussi dans l'application, pour qu'elle soit autonome, en particulier quand on la débogue avec une sonde SWD : le débogueur peut alors la démarrer

directement, sans passer par le chargeur. Copier `ObCP_OTA_Demo/Core/Inc/obcp_ota_request.h` dans le dossier `Core/Inc` du projet, puis dans `main.c` :

```
/* USER CODE BEGIN Includes */
#include "obcp_ota_request.h"
/* USER CODE END Includes */
...
int main(void)
{
  /* USER CODE BEGIN 1 */
  obcp_ota_init_vectors();      /* VTOR = adresse de NOTRE table des vecteurs */
  /* USER CODE END 1 */

  HAL_Init();
```

`obcp_ota_init_vectors()` utilise l'étiquette `g_pfnVectors` du fichier de démarrage : elle donne toujours la bonne adresse, quel que soit l'endroit où l'éditeur de liens a placé le programme.

5.3 Facultatif : revenir au chargeur par logiciel

`obcp_ota_enter_loader()` redémarre la carte directement en mode chargement, sans toucher aux boutons. Elle dépose un mot « magique » dans une petite zone de la SRAM2 (la boîte aux lettres, conservée pendant un reset logiciel) puis provoque un reset. Au redémarrage, le chargeur trouve ce mot, l'efface et reste actif. Deux exemples :

Appui long sur USER (extrait de demo.c)

```
if (pressed)
{
    uint32_t duration = now - button_press_time;
    led_show(white);                       /* retour visuel pendant l'appui */
    if (duration >= BUTTON_LONG_PRESS_MS)  /* 2000 ms */
    {
        obcp_ota_enter_loader();           /* ne revient pas */
    }
}
```

Commande « #ota » dans le programme modèle de l'ObCP

Le programme modèle **OBCP_template_2026**, à partir de sa version 2.1.0, est prêt pour le chargement par Bluetooth : rangé en banque 2, il appelle `obcp_ota_init_vectors()` et comprend la commande `#ota` (module `ota_switch.c`). Elle répond `OTA loader...`, éteint les sorties, ferme proprement la connexion Bluetooth, puis appelle `obcp_ota_enter_loader()`, sans jamais bloquer la boucle principale. C'est cette commande que l'application PC envoie elle-même (§ 4.2).

Le programme modèle de 2024 (`OBCP_template_2024`) peut l'apprendre en une branche de plus : dans `Driver_OBCP.c`, la fonction `processCommand()` reconnaît déjà `#rst`, `start timer`…

```
} else if (strncmp((char*)data_buffer, "#rst", 4) == 0) {
    reset_OBCP_functions();
} else if (strncmp((char*)data_buffer, "#ota", 4) == 0) {
    obcp_ota_enter_loader();          /* redémarre dans le chargeur BLE */
} else if ...
```

La commande `#ota`, tapée dans un terminal Bluetooth (nRF Connect, BLE Terminal…) ou dans le terminal USB, fait alors passer la carte en mode chargement, prête pour le transfert suivant. Cette version minimale

redémarre aussitôt : sa réponse ne part pas et le téléphone constate la coupure avec quelques secondes de retard, sans conséquence pour le transfert.

5.4 Vérifier le projet

La commande **ObCP OTA - verifier le projet** (ou `python verif_cubemx.py CHEMIN_DU_PROJET`) contrôle les points de ce chapitre. Exemple avec le programme modèle de 2024, non modifié :

```
Projet OBCP_template_2024 (application chargée par Bluetooth)
  ERREUR    STM32L476RGTX_FLASH.ld : le programme est placé à 0x08000000. Remplacez
            la ligne FLASH par  FLASH (rx) : ORIGIN = 0x8080000, LENGTH = 512K
  ATTENTION main.c : ajoutez obcp_ota_init_vectors(); dans USER CODE BEGIN 1
            (recommandé, indispensable pour déboguer avec une sonde)
Bilan : 1 erreur(s), 1 recommandation(s).
```

Le programme modèle de 2024 a été compilé avec cette seule modification du script `.ld` : il occupe 67 Ko en configuration Debug, ne fait rien dans la SRAM2, et son transfert simulé s'est déroulé correctement (chapitre 9).

5.5 Règles à respecter

- **Taille** : 512 Ko au plus (la banque 2). C'est très large : le programme modèle de l'ObCP en occupe 67 en Debug.
- **SRAM2** : ne pas utiliser les 16 derniers octets (`0x10007FF0` à `0x10007FFF`), qui servent de boîte aux lettres. Les projets STM32CubeIDE n'utilisent pas la SRAM2 par défaut.
- **Flash** : ne pas effacer toute la flash (*Full chip erase*) et ne pas modifier les octets d'option du STM32 (en particulier BFB2, démarrage sur la banque 2) : le chargeur serait effacé ou ignoré.
- **Débogage** : avec une sonde SWD sur le connecteur J17, vérifier que la configuration de débogage n'efface pas toute la flash avant de programmer.

6. Compatibilité avec STM32CubeMX

6.1 Principe

Le chargeur et la démonstration sont des projets CubeMX ordinaires : la configuration du microcontrôleur (broches, horloge, bibliothèque Bluetooth) est décrite dans le fichier `.ioc`, et CubeMX génère le code d'initialisation (`main.c`, `gpio.c`…).

Tout le code ajouté vit dans des **fichiers séparés**, que CubeMX ne touche jamais : le dossier `Loader/` pour le chargeur ; `demo.c`, `demo.h` et `obcp_ota_request.h` pour la démonstration. Les fichiers générés ne contiennent que quelques appels, placés dans les sections `USER CODE BEGIN … / USER CODE END …` que CubeMX recopie à chaque régénération :

Projet, fichier	Section USER CODE	Contenu ajouté
ObCP_BLE_Loader, `main.c`	Includes	`#include "boot.h"`, `#include "loader.h"`
	1 (début de main)	`boot_check_and_jump();`
	2 (après les MX_..._Init)	`loader_init();`
	3 (dans while (1))	`loader_process();`
	Error_Handler_Debug	LED rouge allumée
ObCP_OTA_Demo, `main.c`	Includes	`#include "obcp_ota_request.h"`, `#include "demo.h"`
	1	`obcp_ota_init_vectors();`
	2 et 3	`demo_init();` puis `demo_process();`

Deux réglages se trouvent hors des fichiers générés et sont conservés à la régénération : le dossier `Loader` comme dossier source et `../Loader/Inc` comme chemin d'inclusion (fichier `.cproject`, configurations Debug et Release), et le script d'édition de liens (limité à la banque 1 pour le chargeur, déplacé en banque 2 pour l'application), qu'en principe CubeMX ne réécrit pas ; `verif_cubemx.py` le contrôle de toute façon.

6.2 Régénérer le code

1. Ouvrir le fichier `.ioc`, faire les modifications, puis *Project › Generate Code* (Alt+K).
2. Lancer **ObCP OTA - verifier le projet** (`verif_cubemx.py`) : chaque crochet, chemin et réglage est contrôlé.
3. Compiler.

> **À savoir :** les commentaires ajoutés dans les fichiers générés (`main.c`) sont écrits sans accents : c'est une simple précaution pour que l'encodage des sections USER CODE ne pose jamais de problème. Les commentaires détaillés, accentués, sont dans les fichiers séparés (UTF-8).

6.3 Réglages CubeMX du chargeur à conserver

Le code du chargeur utilise les étiquettes (*User Label*) des broches, par exemple `USER_BUTTON_Pin` : renommer une étiquette dans CubeMX provoque une erreur de compilation, que `verif_cubemx.py` signale aussi.

Broche	Étiquette	Configuration et rôle
PC13	USER_BUTTON	entrée : bouton USER (actif à 0)
PC6, PC8, PC9	LED_RED, LED_GREEN, LED_BLUE	sorties : LED RVB (transistors Q1 à Q3)
PC2	BLE_RST	sortie : reset du module BLE
PB2	BLE_CS	sortie, niveau haut au repos : sélection du module BLE
PB12	BLE_IRQ	EXTI12 sur front montant : interruption du module
PB13, PB14, PB15	BLE_SCK, BLE_MISO, BLE_MOSI	SPI2 maître vers le module BLE
PC3	EEPROM_CS	sortie, niveau haut : l'EEPROM M95640 partage le bus SPI2

Le composant **X-CUBE-BLE1 6.2.4** est configuré comme dans le projet `OBCP_BLE_UART` du dépôt de l'ObCP : BlueNRG-MS, Controller, Utils, HCI_TL *Basic*, interface *UserBoard* (EXTI PB12, SPI2, CS PB2, Reset PC2). L'horloge vient de l'oscillateur interne HSI (16 MHz), multiplié par la PLL jusqu'à 80 MHz.

Les fichiers `.ioc` ont été dérivés de celui de `OBCP_BLE_UART` (CubeMX 6.11.1, STM32Cube FW_L4 V1.18.1), en ne modifiant que les broches et le nom du projet ; les fichiers générés correspondants ont été adaptés à la main. La première régénération dans CubeMX est l'un des points à vérifier lors des premiers essais (§ 9.3).

7. Fonctionnement détaillé

7.1 Carte mémoire

La flash du STM32L476RG (1 Mo) est formée de **deux banques** de 512 Ko, découpées en **pages** de 2 Ko : c'est la plus petite zone que l'on peut effacer. On n'écrit que dans une zone effacée (octets à `0xFF`), par **double mot** de 8 octets aligné.

Les deux banques peuvent travailler en même temps : on peut effacer ou écrire l'une pendant que le programme s'exécute depuis l'autre (*read-while-write*). Le chargeur, qui s'exécute en banque 1, écrit donc la banque 2 sans s'interrompre ni copier de code en RAM, ce qui simplifie beaucoup sa conception.

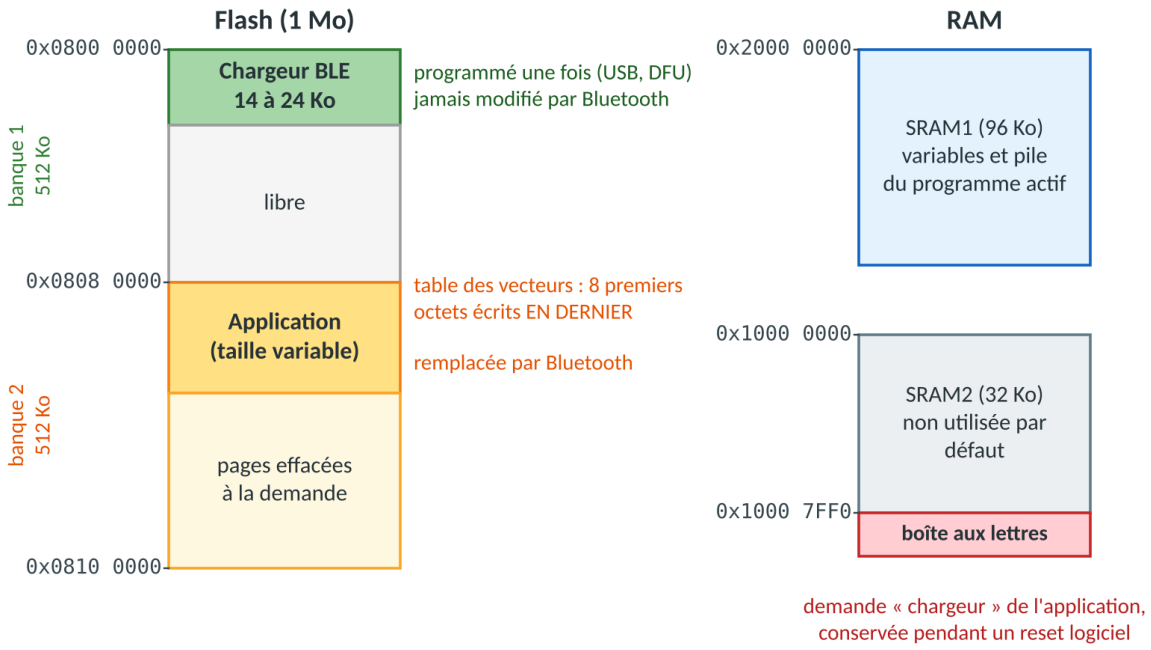


Figure 2 – Répartition de la mémoire (schéma, pas à l'échelle)

Le chargeur occupe 14 Ko (configuration Release) à 24 Ko (Debug) ; son script d'édition de liens est limité à la banque 1, pour qu'il ne puisse jamais déborder sur la zone de l'application. En RAM, le programme actif (chargeur ou application) utilise la SRAM1 ; seuls deux mots de la SRAM2 servent de boîte aux lettres entre l'application et le chargeur.

7.2 Le démarrage : application ou chargeur ?

Après chaque reset, le processeur exécute le chargeur. Sa toute première instruction utile, `boot_check_and_jump()` (fichier `boot.c`), décide en quelques millisecondes de la suite (figure 3). Elle s'exécute **avant** `HAL_Init()`, alors que l'horloge est encore celle du démarrage (MSI à 4 MHz) et qu'aucun périphérique n'est configuré : l'application trouvera ainsi le processeur comme après un vrai reset. C'est pourquoi `boot.c` n'utilise pas la HAL mais accède directement aux registres (RCC, GPIOC, SysTick, SCB).

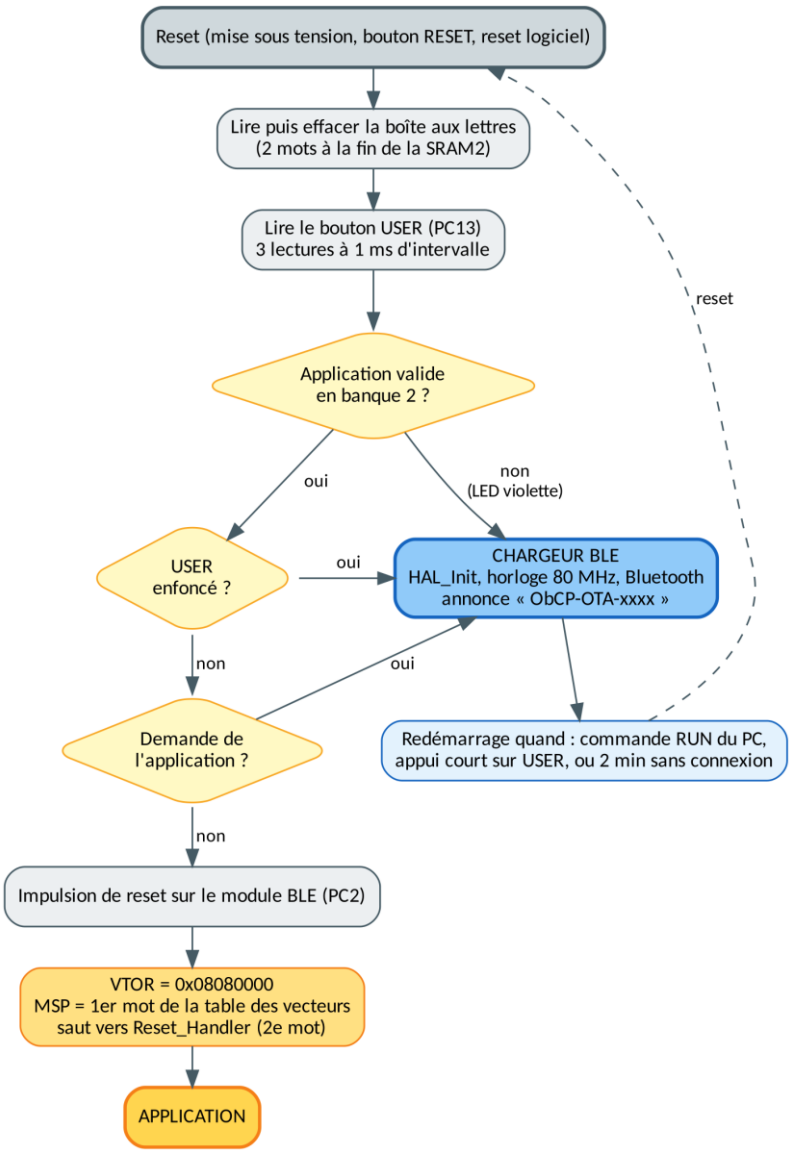


Figure 3 – Décision de démarrage (boot.c) puis fonctionnement du chargeur

Une application est jugée **valide** si les deux premiers mots de la banque 2 ressemblent à une table des vecteurs (`app_image.c`) : le pointeur de pile initial désigne la RAM et est aligné sur 4 octets, et l'adresse de `Reset_Handler` est impaire (instructions Thumb) et située dans la banque 2. Une flash effacée (`0xFFFFFFFF`) n'est donc jamais prise pour une application.

Le saut vers l'application se fait en trois temps : VTOR reçoit l'adresse de la table des vecteurs de l'application, le pointeur de pile (MSP) prend la valeur du premier mot de cette table, puis le processeur saute à l'adresse du second mot (`Reset_Handler`). Les deux dernières opérations sont écrites dans une seule instruction en assembleur : une fois la pile changée, le code C ne doit plus rien y lire.

```
SCB->VTOR = APP_ADDRESS;      /* 0x08080000 : table des vecteurs de l'application */
__DSB();                      /* attend la fin de l'écriture                       */
__ISB();                      /* vide le pipeline du processeur                    */
__asm volatile (
    "msr msp, %0 \n"          /* MSP <- pointeur de pile de l'application  */
    "bx  %1      \n"          /* saut vers Reset_Handler de l'application  */
    : : "r" (initial_sp), "r" (reset_handler) : "memory");
```

Juste avant le saut, le chargeur envoie une impulsion de reset au module Bluetooth. Sans elle, après un transfert, le module continuerait d'annoncer « ObCP-OTA-… » alors que l'application ne s'en sert peut-être pas, ce qui tromperait le PC.

7.3 Le service Bluetooth « ObCP OTA »

En Bluetooth Low Energy, un appareil expose des **services**, qui regroupent des **caractéristiques** : de petites cases de données que le PC peut lire, écrire, ou dont il peut recevoir les changements par **notification**. Chaque élément est identifié par un **UUID** de 128 bits. Le chargeur expose un service à trois caractéristiques :

Élément	UUID	Propriétés	Rôle
service ObCP OTA	`9E090001-33D3-4337-976A-6F024CA6B676`	–	regroupe les trois caractéristiques
CTRL	`9E090002-33D3-…`	écriture, notification	commandes du PC, réponses du chargeur
DATA	`9E090003-33D3-…`	écriture sans réponse	morceaux du programme
INFO	`9E090004-33D3-…`	lecture	renseignements sur le chargeur (18 octets)

Le PC lit INFO avant de commencer, pour vérifier que le chargeur et le programme sont compatibles :

Octets	Contenu
0	version du protocole (1)
1, 2	version du chargeur (majeure, mineure : 1.1)
3	1 si une application valide est installée, 0 sinon
4 à 7	adresse de la zone application (`0x08080000`)
8 à 11	taille maximale d'une application (524 288 octets)
12, 13	taille d'un bloc (1 024 octets)
14	octets de programme par paquet DATA (19)
15	raison du mode chargement : 1 aucune application, 2 bouton USER, 3 demande de l'application
16	diagnostic : cause de la dernière déconnexion, code de la norme Bluetooth (0x08 plus aucun échange radio, 0x13 déconnexion par le PC, 0x16 par la carte) ; 0 si aucune
17	diagnostic : cause du dernier redémarrage imprévu du module Bluetooth (8 plantage, 7 baisse d'alimentation…) ; 0 si aucun

Les octets 16 et 17 existent depuis la version 1.1 du chargeur. L'application PC accepte aussi les 16 octets d'un chargeur 1.0 : elle se passe alors du diagnostic.

Tous les nombres de plusieurs octets sont **petit-boutistes** (*little-endian*) : l'octet de poids faible est transmis en premier, comme le Cortex-M4 les range en mémoire.

7.4 Le protocole de transfert

Le transfert suit toujours la même séquence (figure 4). Les données partent par paquets « sans réponse », les plus rapides en Bluetooth ; tous les 1 024 octets, le PC attend un accusé de réception (ACK) avant de continuer.

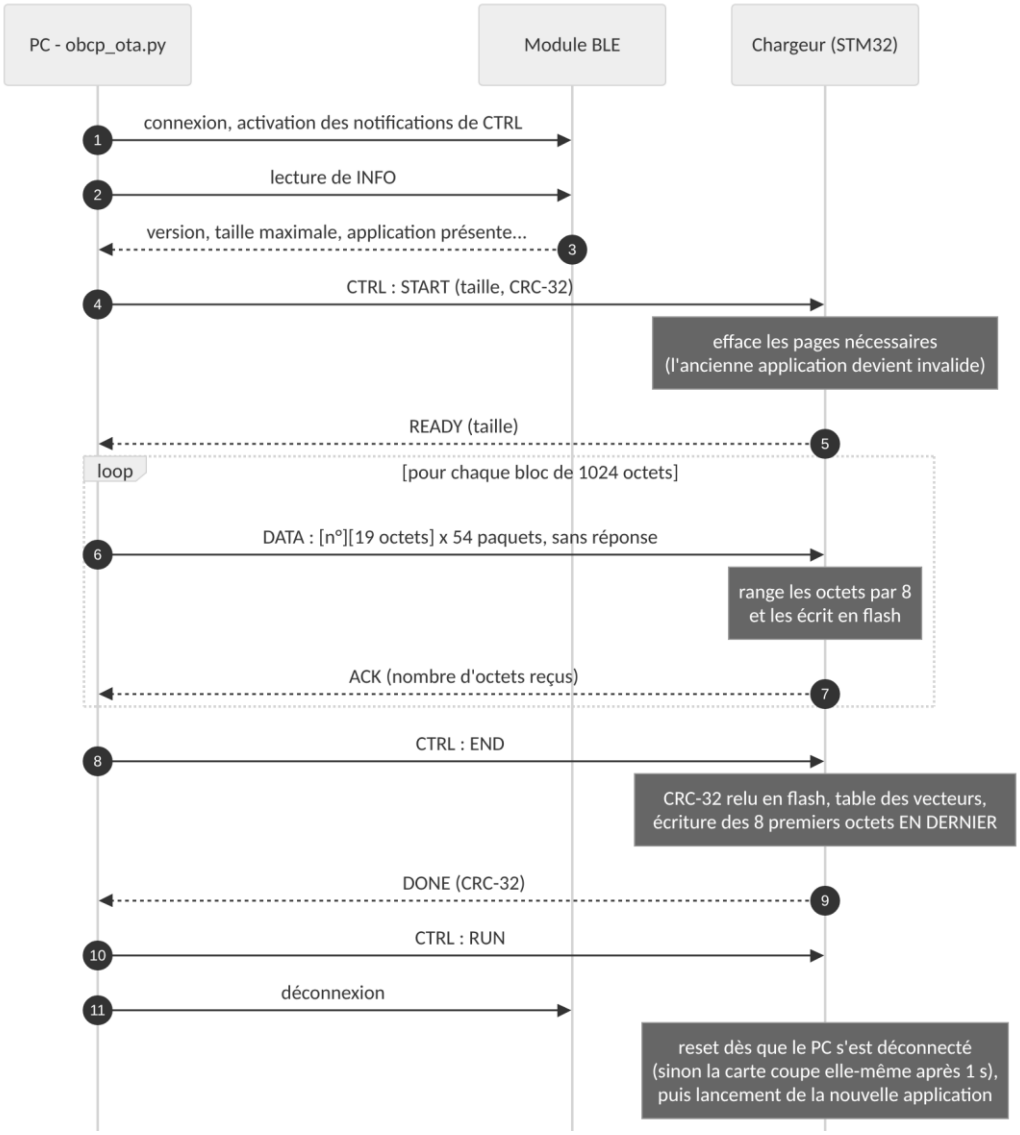


Figure 4 – Déroulement d'un transfert

Les commandes sont écrites par le PC dans CTRL ; le premier octet en est le code :

Commande	Code	Contenu	Réponse
START	0x01	taille (4 octets), CRC-32 attendu (4 octets)	READY, après l'effacement
END	0x02	–	DONE (CRC-32 calculé par la carte)
ABORT	0x03	–	aucune ; retour à l'état IDLE
RUN	0x04	–	aucune ; la carte attend que le PC se déconnecte (1 s au plus), puis redémarre
IDENTIFY	0x05	–	aucune ; LED blanche pendant 3 s

Le chargeur répond par une notification de 5 octets sur CTRL : un code suivi d'une valeur sur 4 octets. READY (0x10) donne la taille acceptée, ACK (0x11) le nombre d'octets reçus, DONE (0x12) le CRC-32 relu en flash. Les erreurs ont les codes suivants :

Code	Nom	Cause	Nouvel essai automatique
0x20	ERR_STATE	commande inattendue à ce moment	non

Code	Nom	Cause	Nouvel essai automatique
0x21	ERR_SIZE	taille de 0 à 7 octets, ou plus de 512 Ko	non
0x22	ERR_FLASH	échec d'effacement ou d'écriture	oui
0x23	ERR_SEQUENCE	paquet perdu (numéro de séquence inattendu)	oui
0x24	ERR_OVERFLOW	plus de données que la taille annoncée	oui
0x25	ERR_CRC	CRC-32 relu en flash différent de celui annoncé	oui
0x26	ERR_INCOMPLETE	END reçu alors qu'il manque des données	oui
0x27	ERR_COMMAND	message mal formé (longueur, code inconnu)	non
0x28	ERR_IMAGE	table des vecteurs invalide (programme mal lié)	non

Un paquet DATA contient un **numéro de séquence** (0 à 255, puis de nouveau 0) suivi de 1 à 19 octets du programme. Le PC découpe chaque bloc de 1 024 octets séparément (53 paquets de 19 octets et un de 17) : un paquet ne chevauche jamais deux blocs, si bien que le nombre d'octets reçus tombe exactement sur un multiple de 1 024 à la fin de chaque bloc.

7.5 Écriture en flash et sûreté de fonctionnement

Le cœur du transfert (`ota.c`) est une **machine à états** (figure 5). Il reçoit des octets, les range par 8 dans un petit tampon et écrit chaque double mot complet en flash.

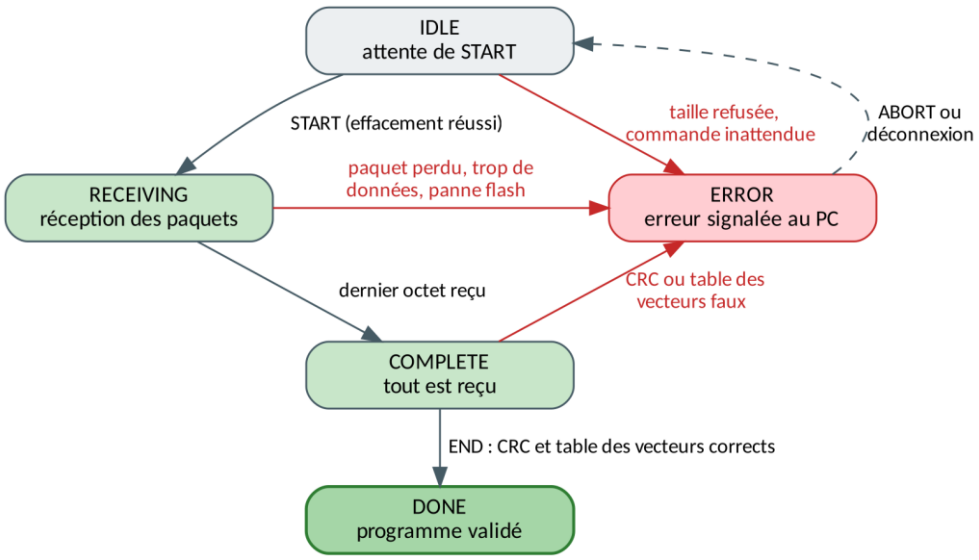


Figure 5 – Machine à états du transfert (ota.c)

Le point clé est l'ordre des écritures : les 8 premiers octets du programme (pointeur de pile et adresse de `Reset_Handler`) sont **gardés en RAM et écrits en dernier**, après la vérification du CRC et de la table des vecteurs. Tant qu'ils ne sont pas écrits, la banque 2 commence par `0xFFFFFFFF` et le chargeur la considère comme vide. Il en découle plusieurs garanties :

- un transfert interrompu (déconnexion, batterie, erreur) laisse une application **invalide**, jamais incomplète : au reset suivant, le chargeur reste actif (LED violette) et il suffit de relancer le transfert ;
- le CRC-32 est recalculé en **relisant la flash**, pas les données reçues : il vérifie ce qui a réellement été écrit ;
- le chargeur ne peut pas être effacé par un transfert : `app_flash.c` n'efface que la banque 2 et refuse toute écriture hors de cette zone. La carte ne peut donc jamais être « bloquée » ;

- avant d'envoyer START (qui efface l'ancienne application), le PC vérifie l'adresse, la taille et la table des vecteurs du nouveau programme : un fichier inadapté n'efface rien ;
- les erreurs passagères (paquet perdu, CRC faux, panne flash) déclenchent automatiquement jusqu'à **trois essais**, chacun repartant de START.

7.6 Contrôle de flux et débit

Le module BlueNRG-MS limite chaque échange à un MTU ATT de 23 octets, soit **20 octets** utiles par écriture : 1 octet de numéro et 19 octets de programme. Le protocole est dimensionné pour cette limite, qui ne peut pas être augmentée sur ce module.

Pourquoi attendre un accusé tous les 1 024 octets ? La bibliothèque X-CUBE-BLE1 range les événements du module (une écriture DATA reçue = un événement) dans une file de **5 places**. Pendant l'envoi d'une commande au module (par exemple la notification ACK), elle peut abandonner des événements si cette file est pleine. Le PC s'arrête donc à la fin de chaque bloc : au moment où le chargeur envoie l'ACK, aucun paquet n'est en route et rien ne peut se perdre. Par précaution, le chargeur vérifie aussi à chaque tour de boucle que la ligne d'interruption du module n'est pas restée active sans être servie (fonction `poll_events()`).

Une fois les notifications activées, le chargeur demande au PC un intervalle de connexion de 15 à 30 ms, pour accélérer les échanges ; le PC reste libre de refuser. L'effacement prend environ 22 ms par page de 2 Ko (0,7 s pour 64 Ko).

> **À savoir :** lors d'un premier essai sur une ObCP (PC sous Windows), les données sont passées à 6,8 Ko/s : environ 1 s pour la démonstration (7 Ko), une dizaine de secondes pour le programme modèle de l'ObCP (67 Ko). Le débit dépend de l'adaptateur Bluetooth du PC (nombre de paquets par intervalle de connexion).

7.7 Nom et adresse Bluetooth de la carte

Comme le programme modèle de l'ObCP, le chargeur lit l'adresse aléatoire statique enregistrée dans chaque module Bluetooth : chaque carte a ainsi une adresse différente, indispensable en salle de TP. Les quatre derniers chiffres hexadécimaux de cette adresse forment le nom annoncé (C1:22:33:44:**A1:B2** donne ObCP-OTA-A1B2).

Le chargeur modifie légèrement le premier octet de l'adresse (C1 devient C0) : le PC voit ainsi deux appareils distincts, le chargeur et l'application, et ne confond pas leurs services, que Windows garde parfois en mémoire. L'annonce contient le nom ; la réponse au balayage (*scan response*) contient l'UUID du service.

L'application PC 1.2 se sert de cette particularité : elle mémorise l'adresse du chargeur de la carte choisie et en déduit celle de son programme, en modifiant le premier octet en sens inverse (`adresse_du_programme()`). Elle reconnaît ainsi la carte quand elle fonctionne normalement, sans la confondre avec ses voisines (§ 4.2). Si le module n'a pas d'adresse enregistrée (cas exceptionnel), le chargeur et le programme modèle en fabriquent chacun une à partir du numéro de série du STM32, mais pas de la même façon : la reconnaissance automatique ne fonctionne pas, il faut passer en mode chargement à la main.

7.8 Fin du transfert et coupures de liaison

Pour le PC, une écriture « avec réponse » (START, END, RUN…) n'est terminée qu'à la réception de son **accusé de réception ATT**, que le module renvoie à l'échange radio suivant, 15 à 30 ms plus tard. Avec la version 1.0, le chargeur coupait la liaison dès la réception de RUN : la coupure pouvait partir avant cet accusé, et Windows

signalait alors une « opération annulée » (WinError -2147023673) alors que le programme était bien installé. C'est très probablement ce qui s'est produit lors du premier essai sur une vraie ObCP. Depuis la version 1.1 :

- le chargeur laisse d'abord le PC se déconnecter, 1 s au plus, avant de couper lui-même (`ble_ota_disconnect()`) ;
- l'application PC considère une coupure pendant RUN comme normale (`envoyer_run()`) ;
- toute autre coupure est signalée aussitôt par Bleak (fonction `disconnected_callback`) et suivie d'une reconnexion : le PC relit INFO, dont l'octet 3 indique si le programme a été validé, et les octets 16 et 17 la cause de la coupure (§ 4.6).

La version 1.1 du chargeur corrige aussi deux fragilités de la liaison entre le STM32 et le module :

- la fonction d'envoi SPI du pilote de ST ne masque pas l'interruption du module : une lecture faite par cette interruption pouvait s'intercaler au milieu d'un envoi et le corrompre. `ble_ota.c` installe une version protégée (`spi_send_guarded()`), sans modifier les fichiers générés par CubeMX ;
- si le module redémarre tout seul (plantage, baisse d'alimentation…), il envoie un événement « module démarré » : le chargeur refait alors toute l'initialisation et redevient visible, au lieu de rester muet.

8. Description du programme

8.1 Le chargeur

Le code du chargeur est découpé en modules indépendants (figure 6), chacun formé d'un fichier `.h` (ce que le module offre aux autres) et d'un fichier `.c` (comment il le fait).

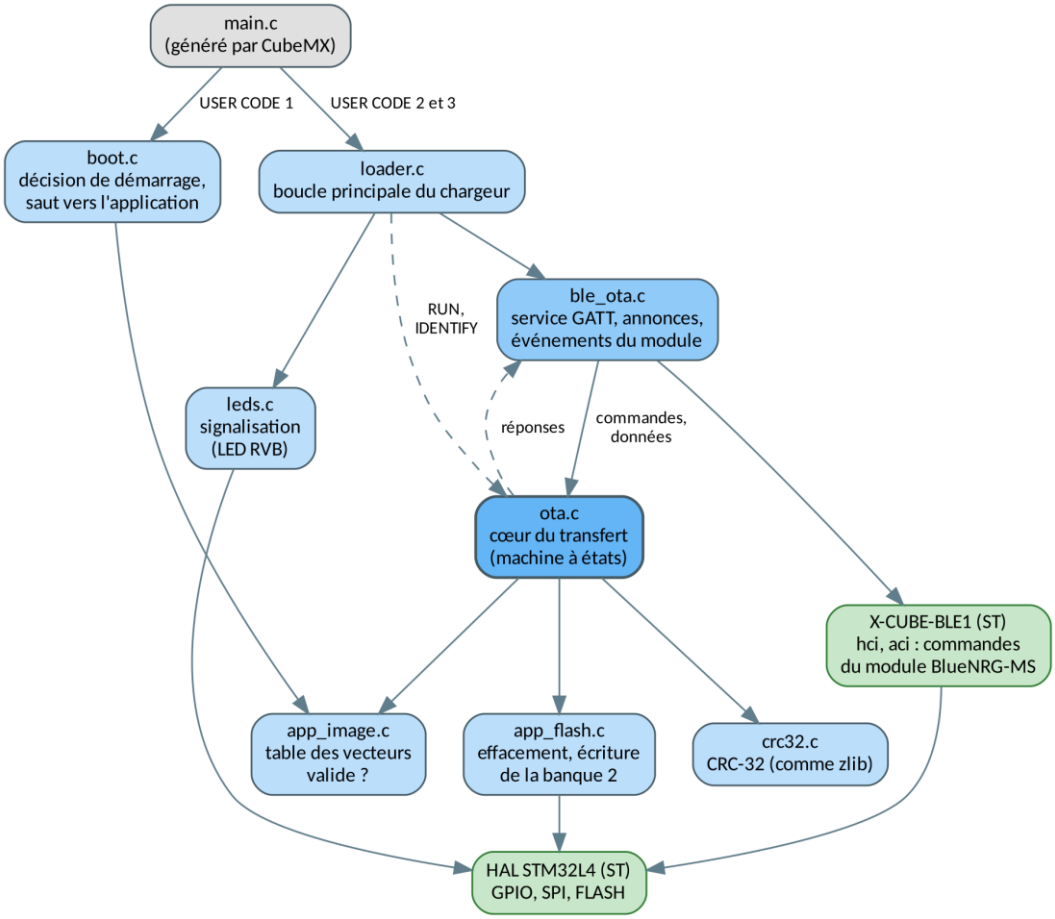


Figure 6 – Modules du chargeur et bibliothèques de ST

Fichier	Rôle
`loader_config.h`	tous les réglages : adresses, nom Bluetooth, temporisations
`ota_protocol.h`	le « contrat » avec l'application PC : UUID, codes, tailles
`boot.c`, `boot.h`	décision de démarrage et saut vers l'application (registres, sans HAL)
`loader.c`, `loader.h`	boucle principale : Bluetooth, LED, bouton, délai d'inactivité, redémarrage
`ble_ota.c`, `ble_ota.h`	module BlueNRG-MS : service GATT, annonces, événements, coupures et diagnostic
`ota.c`, `ota.h`	cœur du transfert : machine à états, écriture, vérifications
`app_flash.c`, `app_flash.h`	effacement et écriture de la banque 2 (HAL FLASH)
`app_image.c`, `app_image.h`	reconnaître une application valide (table des vecteurs)
`crc32.c`, `crc32.h`	CRC-32, identique à celui de zlib et de Python
`leds.c`, `leds.h`	clignotements non bloquants de la LED RVB

Deux choix de conception méritent d'être soulignés :

- Le fichier `ota.c` **ne connaît pas le Bluetooth.** Il reçoit des tableaux d'octets (`ota_on_control()`, `ota_on_data()`) et répond par une fonction qu'on lui donne à l'initialisation (`ota_init(send_response)`, une *fonction de rappel*). Le même fichier est ainsi compilé dans le chargeur et dans les tests sur PC, où le Bluetooth et la flash sont simulés (chapitre 9).
- **La boucle principale ne bloque jamais.** `loader_process()` traite les événements Bluetooth, met à jour les LED, regarde le bouton et les délais, puis rend la main. Aucune attente du type `HAL_Delay()` : on compare l'heure courante (`HAL_GetTick()`) à l'heure d'un événement passé.

8.2 L'application de démonstration

Le projet `ObCP_OTA_Demo` est volontairement minimal : il n'utilise que les GPIO et ne contient aucun code Bluetooth. `demo.c` fait clignoter la LED dans une couleur qui dépend de `DEMO_VERSION`, change la vitesse sur un appui court et appelle `obcp_ota_enter_loader()` sur un appui long. `obcp_ota_request.h` contient les deux fonctions à réutiliser dans vos projets ; tout y est écrit en fonctions `static inline`, si bien qu'il n'y a aucun fichier `.c` à ajouter.

8.3 L'application PC

L'application PC est découpée en quatre modules Python (figure 7). Elle s'appuie sur **Bleak**, une bibliothèque Bluetooth qui fonctionne sous Windows, Linux et macOS.

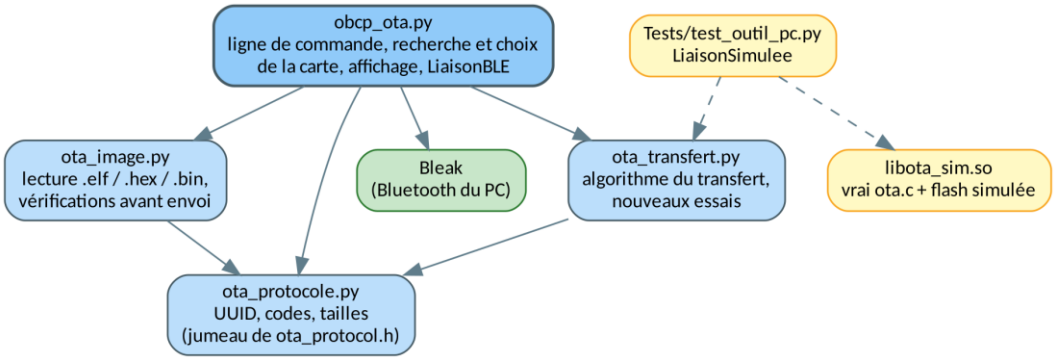


Figure 7 – Modules de l'application PC et des tests

Fichier	Rôle
`obcp_ota.py`	programme principal : options, recherche et choix de la carte, affichage, liaison Bluetooth réelle (`LiaisonBLE`), reconnexion après une coupure, envoi de `#ota` à une carte qui fonctionne (`passer_en_chargement()`)
`ota_image.py`	lecture des fichiers .elf, .hex et .bin ; vérifications avant envoi ; recherche du .elf d'un projet
`ota_transfert.py`	algorithme du transfert (START, blocs, END, RUN), nouveaux essais, détection des coupures (`LiaisonPerdue`) ; indépendant du Bluetooth
`ota_protocole.py`	constantes du protocole, jumelles de `ota_protocol.h` ; fabrication et lecture des messages
`verif_cubemx.py`	vérification d'un projet après régénération par CubeMX (§ 5.4 et 6.2)

Le programme est **asynchrone** (`async` / `await`), comme la bibliothèque Bleak : pendant qu'il attend une réponse de la carte, Python peut traiter les notifications qui arrivent. Comme côté carte, l'algorithme du transfert parle à une « liaison » abstraite (classe `Liaison`) : la vraie liaison Bluetooth dans `obcp_ota.py`, ou une liaison simulée dans les tests, reliée au vrai code C du chargeur.

8.4 Lire le code : quelques notions de C

Le code est commenté pour un lecteur qui découvre le C. Voici les notions qui reviennent le plus souvent :

Notion	Exemple	Signification
`static`	`static uint32_t last_error;`	variable ou fonction visible seulement dans son fichier
`volatile`	`volatile uint32_t *mailbox`	le compilateur relit la valeur à chaque accès (registres, mémoire partagée)
entiers de taille fixe	`uint8_t`, `uint32_t`	entiers non signés de 8 et 32 bits (`stdint.h`)
suffixes `U`, `UL`	`0x08080000UL`	constante non signée (`unsigned long`)
masques de bits	`MODER &= ~(3UL << 26);`	mettre à 0 les bits 26 et 27 d'un registre
pointeur	`const uint8_t *app_flash_read(...)`	adresse en mémoire ; `*p` lit ce qui s'y trouve
structure	`static struct { ... } ota;`	regroupe plusieurs variables sous un même nom
fonction de rappel	`ota_init(send_response);`	on donne une fonction à un module, qui l'appellera plus tard
`_Static_assert`	`_Static_assert(APP_ADDRESS == ...)`	vérification faite par le compilateur, sans coût à l'exécution
`#define`	`#define APP_ADDRESS 0x08080000UL`	nom donné à une constante (remplacé avant la compilation)

9. Tests et état de validation

9.1 Ce qui a été vérifié

Vérification	Résultat
Compilation des deux projets en Debug (-O0) et Release (-Os), avec les options de STM32CubeIDE et des avertissements renforcés pour le code ajouté (-Wextra, -Wshadow…)	0 avertissement (GCC arm-none-eabi 13.2)
Tests unitaires C de `ota.c`, `crc32.c` et `app_image.c` (les vrais fichiers du chargeur) avec une flash simulée qui respecte les règles du STM32L4, sous AddressSanitizer et UBSan : 12 tailles d'images de 8 octets à 512 Ko, toutes les erreurs du protocole, pannes d'effacement et d'écriture, abandon, déconnexion	1 679 vérifications, 0 échec
Tests C de `ble_ota.c` (la partie Bluetooth du chargeur) avec un faux module BlueNRG : initialisation, envoi SPI protégé, transfert, RUN, coupures, événements inhabituels, tampons pleins, redémarrage imprévu du module	66 vérifications, 0 échec
Tests Python : lecture .elf / .hex / .bin, cohérence entre le C et le Python (UUID, codes, adresses, scripts .ld), transferts de bout en bout entre `ota_transfert.py` et le vrai `ota.c` (paquet perdu, octet abîmé, panne flash, carte muette, image mal liée, 512 Ko), programme complet avec un faux module Bleak (choix de la carte, questions, affichage, coupures de liaison et reconnexion, connexion ratée, chargeur 1.0 ; commande `#ota` envoyée à une carte qui fonctionne : carte reconnue parmi d'autres, liaison coupée pendant l'envoi, programme qui ignore `#ota` ou injoignable, fichier de la version 1.1)	52 tests, tous réussis
Programme modèle `OBCP_template_2024` avec seulement le script .ld modifié : compilation, vérifications, transfert simulé avec un paquet perdu	67 Ko, transfert correct au 2e essai automatique
Programme modèle `OBCP_template_2026` 2.1.0 (prêt pour le chargement par Bluetooth) : fichiers .elf, .hex et .bin identiques, transfert simulé avec un paquet perdu ; ses propres tests de la commande `#ota`	55 Ko, transfert correct au 2e essai automatique
`verif_cubemx.py` sur les deux projets	0 erreur, 0 recommandation

9.2 Ce qui reste à essayer sur une vraie ObCP

Un premier essai a été fait sur une vraie ObCP avec la version 1.0, depuis un PC sous Windows : recherche de la carte, connexion, lecture de INFO, effacement, puis envoi d'un programme de 4,6 Ko à 6,8 Ko/s avec tous ses accusés de réception. Une erreur « opération annulée » est apparue à la toute fin, très probablement pendant la commande RUN (§ 7.8) ; la version 1.1 corrige ce point des deux côtés. Restent à valider :

- la fin du transfert avec la version 1.1 (END, RUN, saut vers l'application), le retour au chargeur par la boîte aux lettres et le reset du module ;
- la reconnexion automatique après une coupure, par exemple en éloignant la carte pendant un transfert ;
- l'envoi automatique de `#ota` par l'application PC 1.2, à une carte qui fonctionne avec le programme modèle 2026 (version 2.1) ;
- l'application PC sous Linux et avec d'autres adaptateurs Bluetooth ;
- l'intégration dans STM32CubeIDE (commandes External Tools) et la régénération des deux projets par CubeMX 6.11.

9.3 Procédure de premiers essais

L'ordre ci-dessous isole les éléments un par un ; chaque étape suppose la précédente réussie.

Étape	Action	Résultat attendu, élément validé
1	DFU : programmer `ObCP_BLE_Loader.hex` seul (Full chip erase)	LED violette clignotante : chargeur et module BLE démarrés
2	`python obcp_ota.py --liste`, choisir la carte	carte listée, LED blanche 3 s : annonces, connexion, écriture CTRL
3	`python obcp_ota.py ..\Binaires\ObCP_OTA_Demo_v1.elf`	transfert réussi, LED verte clignotante : flash, CRC, saut
4	appui long sur USER	LED bleue : boîte aux lettres et retour au chargeur
5	transférer `ObCP_OTA_Demo_v2.elf`	LED bleue clignotante (démonstration v2)
6	RESET seul, puis USER + RESET, puis appui court sur USER	application ; chargeur ; application
7	dans CubeIDE : importer, compiler, **ObCP OTA - compiler et transferer**	intégration à l'IDE
8	ouvrir les `.ioc`, *Generate Code*, `verif_cubemx.py`, compiler	compatibilité CubeMX
9	transférer le programme modèle (script .ld modifié) et noter la durée	débit réel
10	pendant un transfert, éloigner la carte ou masquer son antenne	reconnexion automatique, cause affichée
11	programme modèle 2026 (2.1.0) : `#ota` dans un terminal, puis transfert depuis CubeIDE sans toucher la carte	passage au chargeur par `#ota`, envoi automatique par l'application PC

9.4 Relancer les tests

Les tests se lancent sous Linux (ou WSL sous Windows), avec `gcc` et Python 3. Si `arm-none-eabi-gcc` est installé, `run_tests.sh` compile d'abord les deux projets comme STM32CubeIDE ; sinon, il utilise les programmes du dossier `Binaires` :

```
cd Tests
./run_tests.sh                                  compilation (Debug), tests C et Python
python3 build_arm.py ../ObCP_OTA_Demo --release compile un seul projet (Release)
python3 fabriquer_binaires.py                   régénère le dossier Binaires
```

10. Dépannage

Symptôme	Cause probable	Solution
« aucune carte en mode chargement trouvée »	carte pas en mode chargement, trop loin, ou Bluetooth du PC désactivé	USER + RESET (LED bleue) ; activer le Bluetooth du PC ; se rapprocher
« ERREUR (Bluetooth) » au démarrage	pas d'adaptateur Bluetooth, ou adaptateur coupé	activer le Bluetooth (paramètres du système) ; brancher une clé BLE
« la bibliothèque Python bleak n'est pas installée »	Bleak absent pour ce Python	`python -m pip install bleak`
CubeIDE : « Variable references empty selection: ${project_loc} »	aucun projet sélectionné	cliquer sur le projet dans Project Explorer, puis relancer
CubeIDE : « The file does not exist for the external tool »	Python introuvable dans le PATH	mettre le chemin complet de python.exe dans *Location* (§ 3.3)
« le programme commence à l'adresse 0x08000000 »	script .ld non modifié	§ 5.1
« … a été modifié après la dernière compilation réussie »	erreur de compilation	corriger (onglet Problems) ; à défaut `--forcer`
LED rouge fixe pendant le transfert	erreur signalée par la carte	lire le message dans la console ; relancer le transfert
erreurs « paquet perdu » répétées	liaison radio médiocre	se rapprocher ; éviter les obstacles et les clés USB 3.0 près de l'adaptateur
LED rouge, clignotement rapide	le module Bluetooth ne répond pas (soudure, alimentation, SPI2)	vérifier le module ; un appui court sur USER ou 2 min d'attente relancent l'application
LED violette après un transfert interrompu	application effacée mais pas encore remplacée (normal)	relancer le transfert
version 1.0 : « opération annulée » (WinError -2147023673) juste après 100 %	le chargeur 1.0 coupait la liaison dès RUN ; le programme est en général bien installé et lancé	passer à la version 1.1 : application PC, et chargeur en DFU (§ 3.2)
« … n'est pas passée dans son chargeur »	le programme de la carte ne connaît pas la commande `#ota`	USER + RESET, puis relancer ; ou ajouter `#ota` au programme (§ 5.3)
« connexion impossible au programme de la carte … »	carte trop loin, ou un téléphone lui est connecté	déconnecter le téléphone, se rapprocher ; ou USER + RESET
« connexion Bluetooth perdue … nouvelle connexion (2/3) »	liaison radio coupée pendant le transfert	rien à faire, l'application PC reprend ; se rapprocher si cela se répète
« la carte … n'est plus visible après la coupure »	carte redémarrée, éteinte ou trop loin	si le nouveau programme fonctionne, il a été validé ; sinon USER + RESET et relancer
l'application ne démarre pas après le transfert	programme bloqué dès le démarrage (HardFault…), SRAM2 utilisée	essayer la démonstration ; ajouter `obcp_ota_init_vectors()` ; § 5.5
Windows : connexion refusée, « service OTA » introuvable	ancien appairage ou cache Bluetooth de Windows	supprimer l'appareil ObCP-OTA dans les paramètres Bluetooth de Windows, puis relancer
la question posée dans la console ne réagit pas	la console n'a pas le focus	cliquer dans la console de CubeIDE, taper la réponse, puis Entrée

11. Pour aller plus loin

- **Reconnaître la carte sous macOS**, où Bleak ne donne pas les adresses Bluetooth : le programme pourrait par exemple placer dans son annonce un identifiant propre à la carte, que l'application PC comparerait au nom de son chargeur.
- **Sécurité.** En mode chargement, le chargeur accepte un programme de n'importe quel PC à portée : il n'y a ni authentification ni chiffrement. C'est acceptable en TP, puisque le mode chargement demande une action volontaire et s'arrête après 2 minutes sans connexion. Un produit réel vérifierait une **signature** numérique (ECDSA) avant de valider le programme, et chiffrerait l'échange.
- **Double emplacement (A/B).** La banque 2 pourrait accueillir deux applications de 256 Ko : la nouvelle s'écrit à côté de l'ancienne, qui reste disponible en secours si le transfert échoue. Le chargeur s'exécutant en banque 1, aucune astuce de programmation en RAM n'est nécessaire.
- **Débit.** Un module plus récent (BlueNRG-2, Bluetooth 5) accepte des paquets beaucoup plus grands que 20 octets ; le protocole n'aurait qu'à lire la taille des paquets dans INFO, comme il le fait déjà.
- **Mesures.** Le transfert est un bon support de TP pour mesurer un débit, l'effet de l'intervalle de connexion, ou le coût énergétique d'une mise à jour sur batterie.

Annexe A Glossaire

Terme	Définition
ACK	accusé de réception : message qui confirme l'arrivée des données
BLE	Bluetooth Low Energy, version du Bluetooth économe en énergie (depuis Bluetooth 4.0)
Banque (flash)	moitié de la flash du STM32L476 (512 Ko), effaçable et programmable indépendamment de l'autre
Caractéristique	case de données d'un service BLE, que le PC peut lire, écrire ou recevoir en notification
CRC-32	empreinte de 32 bits d'une suite d'octets ; un seul bit changé change l'empreinte
DFU	Device Firmware Upgrade : programmation par l'USB, avec le chargeur gravé par ST dans le STM32
Double mot	8 octets : la plus petite quantité que l'on écrit en flash sur le STM32L4
Éditeur de liens	outil qui place le code et les données aux adresses définies par le script .ld
GATT	organisation des données d'un appareil BLE en services et caractéristiques
HAL	bibliothèque de ST qui simplifie l'accès aux périphériques (HAL_GPIO_WritePin…)
MTU	taille maximale d'un échange ATT (23 octets sur le BlueNRG-MS, soit 20 octets utiles)
Notification	envoi spontané d'une valeur de caractéristique par l'appareil BLE vers le PC
OTA	Over The Air : mise à jour d'un programme sans fil
Page (flash)	plus petite zone effaçable : 2 Ko sur le STM32L476
Petit-boutiste	ordre des octets d'un nombre : poids faible en premier (*little-endian*)
Table des vecteurs	début de tout programme Cortex-M : pointeur de pile initial puis adresses des fonctions d'interruption
UUID	identifiant unique de 128 bits d'un service ou d'une caractéristique BLE
VTOR	registre du Cortex-M qui indique l'adresse de la table des vecteurs

Annexe B Référence des fonctions

Liste extraite automatiquement des commentaires du code. Les fonctions marquées *interne* sont déclarées `static` : elles ne servent qu'à l'intérieur de leur fichier.

B.1 Chargeur et démonstration (C)

boot.c – décision de démarrage

Fonction	Rôle
`boot_check_and_jump()`	Lance l'application si rien ne s'y oppose ; sinon revient.
`boot_get_reason()`	Raison du maintien dans le chargeur (après boot_check_and_jump).
`boot_delay_ms()` *interne*	Attente active de 'ms' millisecondes avec le compteur SysTick.
`boot_mailbox_take()` *interne*	Lit la « boîte aux lettres » de la SRAM2, puis l'efface.
`boot_button_pressed()` *interne*	Lit le bouton USER (PC13).
`boot_reset_ble_module()` *interne*	Impulsion de reset sur le module BLE (PC2 à 0 pendant 5 ms).
`boot_jump_to_application()` *interne*	Saut vers l'application.

loader.c – boucle principale

Fonction	Rôle
`loader_init()`	Démarre le Bluetooth et prépare l'affichage.
`loader_process()`	Un tour de boucle : Bluetooth, LED, bouton, délai d'inactivité, redémarrage demandé par le PC.
`loader_restart()` *interne*	Redémarre la carte proprement : LED éteintes, connexion coupée, reset.
`button_short_press()` *interne*	Détection d'un appui court (enfoncé au moins 50 ms puis relâché).
`update_leds()` *interne*	Choisit l'affichage des LED d'après l'état du Bluetooth et du transfert.

ble_ota.c – Bluetooth

Fonction	Rôle
`ble_ota_init()`	Initialise le module BLE et crée le service « ObCP OTA ».
`ble_ota_process()`	Traite les événements BLE et relance les annonces si besoin.
`ble_ota_is_ready()`	Indique si le module BLE est initialisé et répond.
`ble_ota_is_connected()`	Indique si un PC est connecté.
`ble_ota_disconnect()`	Termine proprement la connexion en cours, avant un redémarrage.
`ble_ota_get_name()`	Nom annoncé par la carte, par exemple « ObCP-OTA-A1B2 ».
`spi_send_guarded()` *interne*	Envoie une commande au module, à l'abri de l'interruption du module.
`use_guarded_spi_send()` *interne*	Remplace la fonction d'envoi du pilote par spi_send_guarded() (les autres fonctions restent celles du pilote).
`refresh_info()` *interne*	Met à jour la caractéristique INFO (lue par le PC avant le transfert).
`send_response()` *interne*	Envoie une réponse au PC : notification de 5 octets sur CTRL.
`address_is_blank()` *interne*	Une adresse « vide » (que des 0 ou que des 0xFF) n'est pas utilisable.
`choose_board_address()` *interne*	Choisit l'adresse BLE de la carte.
`build_name()` *interne*	Construit le nom "ObCP-OTA-XXXX" : XXXX = deux derniers octets de l'adresse, tels qu'affichés à la fin de « C1:22:33:44:A1:B2 ».

Fonction	Rôle
`add_ota_service()` *interne*	Crée le service et ses trois caractéristiques dans le module.
`start_advertising()` *interne*	Lance les annonces : la carte devient visible et connectable.
`connection_ended()` *interne*	La connexion vient de se terminer (le PC s'est déconnecté, ou la liaison radio est tombée) : on note la cause pour le diagnostic, on abandonne le transfert en cours et on redevient visible.
`on_attribute_modified()` *interne*	Une écriture du PC vient d'arriver dans une de nos caractéristiques.
`ble_event_handler()` *interne*	Fonction appelée par la bibliothèque (hci_user_evt_proc) pour chaque événement reçu du module.
`poll_events()` *interne*	Lit les événements en attente et les traite.
`wait_disconnection()` *interne*	Attend la fin de la connexion (événement EVT_DISCONN_COMPLETE), au plus timeout_ms millisecondes, en continuant à traiter les événements.

ota.c – cœur du transfert

Fonction	Rôle
`ota_init()`	Prépare le module (à appeler une fois au démarrage).
`ota_on_control()`	Traite une commande écrite par le PC dans la caractéristique CTRL.
`ota_on_data()`	Traite un paquet écrit par le PC dans la caractéristique DATA.
`ota_on_disconnect()`	À appeler quand le PC se déconnecte : un transfert inachevé est abandonné (l'application reste invalide tant qu'un nouveau transfert n'a pas réussi).
`ota_get_state()`	Renvoie l'état courant du transfert.
`ota_get_received()`	Nombre d'octets du programme reçus depuis START.
`ota_get_size()`	Taille annoncée par START (0 si aucun transfert).
`ota_take_run_request()`	Vrai une seule fois après réception de la commande RUN.
`ota_take_identify_request()`	Vrai une seule fois après réception de la commande IDENTIFY.
`ota_take_activity()`	Vrai une seule fois après chaque accusé de réception envoyé (sert à faire clignoter la LED verte au rythme du transfert).
`ota_build_info()`	Remplit la caractéristique INFO (OTA_INFO_LEN octets, voir ota_protocol.h).
`read_u32()` *interne*	Lit un entier de 32 bits petit-boutiste (octet de poids faible en premier).
`write_u32()` *interne*	Écrit un entier de 32 bits en petit-boutiste.
`respond()` *interne*	Envoie une réponse au PC (si une fonction de réponse est enregistrée).
`reset_session()` *interne*	Remet à zéro les compteurs d'un transfert (sans toucher à la flash).
`fail()` *interne*	Passe en erreur : reverrouille la flash et prévient le PC.
`store_dword()` *interne*	Range le double mot complet : en RAM s'il s'agit du premier (il sera écrit en dernier), en flash sinon.
`handle_start()` *interne*	START : [0x01][taille sur 4 octets][CRC-32 sur 4 octets].
`handle_end()` *interne*	END : toutes les données sont arrivées, on vérifie puis on valide.

app_flash.c – flash de la banque 2

Fonction	Rôle
`app_flash_erase()`	Déverrouille la flash et efface les pages qui contiendront 'size' octets à partir du début de la zone application.
`app_flash_program()`	Écrit un double mot (8 octets) dans la zone application.
`app_flash_read()`	Donne l'adresse en mémoire d'un octet de la zone application.

Fonction	Rôle
`app_flash_lock()`	Reverrouille la flash (à appeler à la fin ou en cas d'erreur).
`app_flash_flush_caches()`	Vide les mémoires cache de la flash.
`app_flash_last_error()`	Code d'erreur de la dernière opération ratée (HAL_FLASH_GetError()).

app_image.c – application valide ?

Fonction	Rôle
`app_image_vectors_ok()`	Teste si les deux premiers mots d'une image ressemblent à une table des vecteurs prévue pour la zone application.
`app_image_is_installed()`	Indique si une application valide est installée en banque 2.
`read_u32()` *interne*	Lit un mot de 32 bits rangé en petit-boutiste à l'adresse p.

crc32.c – CRC-32

Fonction	Rôle
`crc32_update()`	Poursuit le calcul d'un CRC-32 avec de nouveaux octets.

leds.c – LED RVB

Fonction	Rôle
`leds_set_mode()`	Choisit le mode d'affichage (sans effet s'il est déjà actif).
`leds_pulse()`	Inverse la LED verte (appelé à chaque bloc reçu).
`leds_process()`	Met à jour les LED ; à appeler dans la boucle principale.
`leds_write()` *interne*	Allume ou éteint chacune des trois couleurs.

demo.c – démonstration

Fonction	Rôle
`demo_init()`	Prépare la démonstration (LED éteintes, chronomètres à zéro).
`demo_process()`	Un tour de boucle : clignotement et bouton.
`led_show()` *interne*	Allume la LED RVB dans la couleur demandée.
`button_is_pressed()` *interne*	Le bouton USER relie PC13 à la masse : enfoncé = niveau bas (0).

obcp_ota_request.h – fonctions pour les applications

Fonction	Rôle
`obcp_ota_init_vectors()`	Fait pointer le registre VTOR sur la table des vecteurs de l'application.
`obcp_ota_enter_loader()`	Redémarre la carte dans le chargeur BLE.

B.2 Application PC (Python)

obcp_ota.py – programme principal

Fonction	Rôle
`journal()`	Affiche une ligne (avec un retrait), immédiatement.
`etape()`	Affiche le titre d'une des 4 étapes, par exemple « [2/4] Recherche.
`nombre()`	12345 -> '12 345' (écriture française).
`decimal()`	3.14 -> '3,1' (virgule décimale).
`Progression` *classe*	Barre de progression.
`demander()`	Pose une question ; renvoie None si personne ne peut répondre.
`lire_memoire()`	Contenu du fichier ~/.obcp_ota.json : {"carte": nom de la carte, "adresse": adresse Bluetooth de son chargeur}, ou {} s'il n'existe pas.
`lire_carte_memorisee()`	Nom de la carte choisie la dernière fois, ou None.
`memoriser_carte()`	Enregistre la carte choisie et l'adresse Bluetooth de son chargeur (ou oublie tout si nom vaut None).
`normaliser_nom()`	'a1b2', 'A1B2' ou 'ObCP-OTA-A1B2' -> 'ObCP-OTA-A1B2'.
`importer_bleak()`	Importe Bleak, ou explique comment l'installer.
`chercher_cartes()`	Écoute les annonces Bluetooth et renvoie {nom: (appareil, signal_dBm)}.
`lister()`	Affiche les cartes, de la plus proche (signal le plus fort) à la plus loin.
`choisir()`	Décide quelle carte utiliser ; pose une question si c'est ambigu.
`LiaisonBLE` *classe*	Liaison réelle : caractéristiques du service OTA par Bleak.
`LiaisonBLE.demarrer()`	Vérifie la présence du service OTA et active les notifications de CTRL.
`LiaisonBLE._notification()`	Appelée par Bleak à chaque notification de CTRL.
`LiaisonBLE.lire_info()`	Lit la caractéristique INFO.
`LiaisonBLE.ecrire_commande()`	Écrit une commande dans CTRL, avec accusé de réception.
`LiaisonBLE.ecrire_donnees()`	Écrit un paquet dans DATA, sans réponse.
`connecter()`	Connexion à la carte.
`deconnecter()`	Ferme la connexion Bluetooth (sans erreur si la carte est déjà partie).
`preparer_image()`	Étape 1 : trouve et vérifie le programme à envoyer.
`passer_en_chargement()`	La carte visée fonctionne normalement (son programme annonce) : on lui envoie la commande « #ota », comme on la taperait dans un terminal.
`retrouver_carte()`	Après une coupure, attend que la carte réapparaisse : le chargeur se remet à annoncer dès qu'il constate la déconnexion.
`afficher_diagnostic()`	Après une reconnexion : ce que la carte a noté sur la coupure (chargeur 1.1 ; un chargeur 1.0 ne fournit pas ces renseignements).
`commande_transfert()`	Étapes 2 à 4 : trouver la carte, s'y connecter, transférer le programme.
`commande_liste()`	Option --liste : affiche les cartes en mode chargement et mémorise la sienne.
`analyser_arguments()`	Lit les options de la ligne de commande (module argparse).
`main()`	Programme principal ; renvoie 0 si tout s'est bien passé, 1 sinon.

ota_image.py – lecture des fichiers

Fonction	Rôle
`ErreurImage` *classe*	Le fichier ne peut pas être envoyé (le message explique pourquoi).

Fonction	Rôle
`Image` *classe*	Programme prêt à être envoyé.
`Image.taille()`	Nombre d'octets du programme.
`Image.crc32()`	CRC-32 de l'image, identique à celui que calcule l'ObCP.
`lire_elf()`	Extrait les segments à charger d'un fichier ELF 32 bits (ARM).
`lire_hex()`	Lit un fichier Intel HEX.
`assembler()`	Met les morceaux bout à bout.
`charger()`	Lit un fichier .elf, .hex ou .bin et renvoie l'image correspondante.
`verifier()`	Lève ErreurImage si le programme ne peut pas fonctionner une fois chargé en banque 2.
`trouver_elf()`	Cherche le fichier .elf le plus récent dans les dossiers de compilation du projet (Debug, Release ou tout autre sous-dossier).
`source_plus_recente()`	Renvoie un fichier source modifié APRÈS la création du .elf (la dernière compilation a donc échoué ou n'a pas été lancée), ou None.
`decrire_date()`	Date de modification d'un fichier, lisible (pour les messages).

ota_transfert.py – algorithme du transfert

Fonction	Rôle
`ErreurTransfert` *classe*	Échec du transfert.
`LiaisonPerdue` *classe*	La liaison Bluetooth a été coupée.
`Liaison` *classe*	Interface commune aux liaisons (Bluetooth réel ou simulation).
`Liaison.lire_info()`	Lit la caractéristique INFO (16 octets avec un chargeur 1.0, 18 avec un 1.1).
`Liaison.ecrire_commande()`	Écrit une commande dans CTRL (écriture avec accusé de réception).
`Liaison.ecrire_donnees()`	Écrit un paquet dans DATA (écriture sans réponse, plus rapide).
`Liaison.signaler_coupure()`	À appeler dès que la liaison est coupée : une attente de réponse en cours s'arrête aussitôt (au lieu d'attendre la fin de son délai).
`Liaison.recevoir_notification()`	À appeler à chaque notification reçue sur CTRL.
`Liaison.vider_reponses()`	Oublie les réponses arrivées et non lues.
`Liaison.attendre_reponse()`	Attend la prochaine réponse de la carte : (code, valeur).
`erreur_de_la_carte()`	Transforme une réponse inattendue de la carte en message clair.
`lire_et_verifier_info()`	Lit INFO et vérifie que la carte peut recevoir ce programme.
`_un_essai()`	Un transfert complet : START, données, END.
`transferer()`	Envoie le programme, avec de nouveaux essais en cas d'erreur passagère.
`envoyer_run()`	Demande à la carte de redémarrer sur le nouveau programme (RUN).
`identifier()`	Demande à la carte de faire clignoter sa LED en blanc (3 s).

ota_protocole.py – constantes et messages

Fonction	Rôle
`commande_start()`	Message START : code, taille du programme, CRC-32 attendu.
`commande_simple()`	Message d'un seul octet (END, ABORT, RUN, IDENTIFY).
`paquet_donnees()`	Paquet DATA : numéro de séquence (0 à 255) suivi de 1 à 19 octets.
`decoder_reponse()`	Notification CTRL -> (code, valeur).

Fonction	Rôle
`nom_reponse()`	Nom lisible d'un code de réponse, pour les messages.
`InfoChargeur` *classe*	Contenu de la caractéristique INFO, décodé.
`decoder_info()`	16 ou 18 octets de INFO -> InfoChargeur (voir ota_protocol.h).
`decrire_cause_deconnexion()`	Texte lisible pour l'octet 16 de INFO.
`decrire_redemarrage_module()`	Texte lisible pour l'octet 17 de INFO.
`adresse_du_programme()`	Adresse Bluetooth du programme d'une carte, d'après celle de son chargeur.

verif_cubemx.py – vérification d'un projet

Fonction	Rôle
`ok()`	Enregistre un point vérifié avec succès.
`attention()`	Enregistre une recommandation (le projet fonctionnera quand même).
`erreur()`	Enregistre une erreur (le projet ne fonctionnera pas).
`lire()`	Contenu d'un fichier texte, ou None s'il n'existe pas.
`section()`	Contenu d'une section /* USER CODE BEGIN nom */ .
`sans_commentaires()`	Retire les commentaires C (pour ne pas confondre un appel et un commentaire).
`zone_flash()`	(origine, taille) de la zone FLASH du script d'édition de liens.
`verifier_chargeur()`	Vérifications propres au projet du chargeur (ObCP_BLE_Loader).
`verifier_application()`	Vérifications pour une application chargée par Bluetooth.
`main()`	Programme principal ; renvoie 0 sans erreur, 1 sinon, 2 si mal appelé.